



Lab Report

Course Name: Problem Solving and Programming Experiment

Student Name: Li Rongyao

Student ID: 2024103975

College: International School

Major: CST

Office of Academic Affairs, Jinan University

2025/06/17

LAB REPORT 实验报告

Lab Title	Data Types and Variables			Lab No.	01
Stud. Name	LiRongyao	Major	CST	Class	2024CST
Student ID	2024103975		Date	3/3/2025	

Lab description/objectives:

Description:

Modify example to convert a Celsius temperature into its Fahrenheit equivalent.

By using the formula, write a C program to calculate.

Objectives:

By completing the tasks to understand the using of data types and variables. From the first task, we could better choose the appropriate data types and understand the output of double type. From the second task, we could better use the arithmetic method to ensure the accuracy of the result.

Source code:

Task 1:

```
#include <stdio.h>

int main()
{
    double celsius = 40.5;
    double fahrenheit ; /* declaration and initialization */

    fahrenheit = (celsius * 9.0 / 5.0) + 32.0;
    printf("The Fahrenheit equivalent of %5.2f degrees Celsius\n",
           celsius);
    printf("    is %5.2f degrees\n", fahrenheit);

    return 0;
}
```

Task 2:

```
#include <stdio.h>

int main(){
    int upper = 20;
```

```
int lower = 9;

int sum = 0;

sum = (upper*(upper+1)/2)*(upper*(upper+1)/2)-(lower*(lower+1)/2)*(lower*(lower+1)/2);

printf("The sum of cubes from 10 to 20 is %d", sum);

return 0;

}
```

Program outputs:

Task 1:

```
The Fahrenheit equivalent of 40.50 degrees Celsius
is 104.90 degrees
```

Task 2:

```
The sum of cubes from 10 to 20 is 42075
```

Discussion:

1. **Most difficult parts**

Clearly realize the relationship between 2 degrees, use correct expression to calculate.

2. **Bugs and/or Errors**

Might forgot to use float number in the expression, finally caused the program had a wrong output.

LAB REPORT

Lab Title	Interactive Input and Selections	Lab No.	2			
Student Name	LiRongyao	Student ID	2024103975			
Date	2025/3/10					
Lab description/objectives: 1. Demonstrate understanding of interactive input handling, selection statement (if-else, switch) and logical operators in C programming. 2. Implement percentage-to-grade conversion and conditional message printing based on character input as defined in the lab tasks/requirements. 3. Analyze input validation, handling of invalid entries, and proper use of conditional logic to ensure program robustness.						
Source code: Task 1: <pre>#include <stdio.h> int main(){ double i; int temp = scanf("%lf", &i); if (!temp){ printf("Invalid Value"); return 0; } if (i<60){ printf("Your grade is F."); }else if (i<70){ printf("Your grade is D."); }else if (i<80){ printf("Your grade is C."); }else if (i<90){ printf("Your grade is B."); }else if (i<=100){ printf("Your grade is A."); }else { printf("Invalid value."); } return 0; }</pre> Task 2: <pre>#include <stdio.h> #include <ctype.h></pre>						

```
int main(){
    char score;
    scanf("%c", &score);
    score = toupper(score);
    switch (score)
    {
    case 'A':
    case 'B':
        printf("Excellent!");
        break;
    case 'C':
    case 'D':
        printf("Good");
        break;
    case 'E':
    case 'F':
        printf("Need Improvement");
        break;
    case 'O':
        printf("Outstanding!");
        break;
    case 'U':
        printf("Undefined grade");
        break;
    default:
        printf("Invalid grade");
        break;
    }
    return 0;
}
```

Program output:

Task 1:

```
78
Your grade is C.
```

```
r
Invalid Value
```

```
180
Invalid value.
```

Task 2:

```
a  
Excellent!
```

```
0  
Outstanding!
```

```
10  
Invalid grade
```

Analysis:

1. Validate Range of Inputs

- The range of task 1 is 0-100. If we input the number out the range or input other character, the input is invalid.
- The range of task 2 is those letters in either uppercase or lowercase. If we input others then the input is invalid.

2. Optimize

- In task 1 we choose the right scope to decrease the number of judgements.
- In task 2 we do not use “break” in some cases and use the function to transfer the lowercase letter to uppercase letter to get our wanted result.

3. Manage Edge Cases:

- In task 1, we check the return value of the function “scanf()” to ensure the right input. Then we eliminate those invalid cases.
- In task 2, we set default case as the invalid input to ensure the right input.

Conclusion:

- When doing the interactive input, we need to ensure the right format of the input and cope with the input in advance.
- When doing selection, we need to set the appropriate scope and optimize the cases so that we can get the right output more accurately and rapidly.

LAB REPORT

Lab Title	Loops	Lab No.	03
Student Name	Li Rongyao	Student ID	2024103975
Date	2025/3/17		
Lab description/objectives: - Demonstrate understanding of key C programming concepts including for loop, while loop, arithmetic operations, conditional statements, basic input/output, and data types. - Implement two specific functionalities/algorithms: - Calculate the factorial of a given number using a for loop. - Find the sum of all digits of a given integer using a while loop. - Analyze critical aspects.			
Source code: Task 1: <pre>#include <stdio.h> int main(){ int n; int ans = 1; printf("Enter a positive integer.\n"); scanf("%d", &n); for (int i = 1; i <= n; i++){ ans *= i; } printf("The product from 1 to %d is %d.\n", n, ans); return 0; }</pre> Task 2: <pre>#include <stdio.h> int main(){ int n; int sum = 0; printf("Enter a positive number.\n"); scanf("%d", &n); int temp = n; while (temp != 0){ sum += temp % 10; temp /= 10; } printf("The sum of the digits of %d is %d.\n", n, sum); return 0; }</pre>			

```
}
```

Program output:

Task 1:

```
Enter a positive integer.  
5  
The product from 1 to 5 is 120.
```

```
Enter a positive integer.  
1  
The product from 1 to 1 is 1.
```

Task 2:

```
Enter a positive number.  
1234  
The sum of the digits of 1234 is 10.
```

```
Enter a positive number.  
0  
The sum of the digits of 0 is 0.
```

Analysis:

1. The For Loops to Calculate the Factorial

- Ensure the terminal condition and the corresponding expression as test condition in the for loop. In this task is to multiple from 1 to n so the terminal condition is $i = n$. And the corresponding test condition is $i \leq n$.

- Fulfill the loop body by setting the initial value of product is 1 and then multiple the integers one by one.

2. The While Loops to Find the Sum

- Ensure the method to find the answer. We need to split the digits one by one by taking remainders and then modify the integer by division.

- Find the corresponding test condition and loop body. The loop will halt as the digit has completely split thus the test condition is the integer not equals to 0. Then we need to get the sum first by add the digit with the before sum and then modify the integer by dividing 10.

Conclusion:

For the loops, we need to ensure the terminal condition or test condition first after setting the method to

solve the problem. Then we need to use appropriate operations in the loop body to fulfill the goal.

LAB REPORT

Lab Title	Functions	Lab No.	04
Student Name	Li Rongyao	Student ID	2024103975
Date	2025/3/24		

Lab description/objectives:

1. Demonstrate understanding of function implementation, random number generation, loop structures in C programming.
2. Implement a simple guessing game using the rand() function and write functions to calculate the greatest common divisor (GCD) and least common multiple (LCM) of two integers as defined in the lab tasks.
3. Analyze critical aspect.

Source code:

Task 1:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
void guessgame(int n,int target){
    if (n > target){
        printf("Too high! Try again.\n");
    } else if(n < target){
        printf("Too low! Try again.\n");
    } else {
        printf("Congratulations! You guessed it right.\n");
    }
}
int main(){
    srand(time(0));
    int upperbound = 100;
    int target = 1 + (int)rand() % upperbound;
    int n;
    printf("Please Enter your guess.\n");
    do {
        scanf("%d", &n);
        guessgame(n, target);
    } while (n != target);
    return 0;
}
```

Task 2:

#include <stdio.h>

```
int gcd(int a, int b){
    int temp;
    if (a < b){
        temp = a;
        a = b;
        b = temp;
    }
    while (b != 0){
        temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

int lcm(int a, int b){
    if (a*b == 0){
        return 0;
    }else {
        return (a * b) / gcd(a, b);
    }
}

int main(){
    int a, b;
    printf("Please Enter the two number you want to find the GCD and LCM.\n");
    scanf("%d%d", &a, &b);
    printf("The gcd of %d and %d is %d.\n", a, b, gcd(a, b));
    printf("The lcm of %d and %d is %d.\n", a, b, lcm(a, b));
    return 0;
}
```

Program output:

Task 1:

```
Please Enter your guess.  
50  
Too low! Try again.  
75  
Too low! Try again.  
87  
Too low! Try again.  
94  
Too high! Try again.  
90  
Congratulations! You guessed it right.
```

Task 2:

```
Please Enter the two number you want to find the GCD and LCM.  
15 12  
The gcd of 15 and 12 is 3.  
The lcm of 15 and 12 is 60.  
Please Enter the two number you want to find the GCD and LCM.  
0 12  
The gcd of 0 and 12 is 12.  
The lcm of 0 and 12 is 0.
```

Analysis:

1. Guess game

- Learn about the function of producing random number and corresponding head file.
- Cope with the random number in the appropriate scope.
- Write a function and loop to perform the guess game.

2. GCD and LCM

- Use the right algorithm in the function to find the GCD.
- Note the order of the two number and the two function to find GCD and LCM.
- Be cautious about the 0 in the function.

Conclusion:

For the functions, we need to have the ability to learn about the functions in the standard library and to write the functions by ourselves to produce some results or perform some operations. And we should master the elements in the function like the name, parameters and so on.

LAB REPORT

Lab Title	Pointers and Recursion	Lab No.	05
Student Name	Li Rongyao	Student ID	2024103975
Date	2025/3/31		

Lab description/objectives:

1. Demonstrate understanding of pointers and recursion in C programming.
2. Implement the functionality of extracting phone-number components from a combined integer and calculating Tribonacci numbers as defined in the lab tasks/requirements.
3. Analyze the critical aspects.

Source code:

Task I:

#include <stdio.h>

void phoneDecompose(int *number*, int* *areaCode*, int* *exchangeCode*, int* *lineNumber*){

int temp1 = 1e7;

int temp2 = 1e4;

 **areaCode* = *number* / temp1; *number* %= temp1; **exchangeCode* = *number* / temp2; **lineNumber* = *number* % temp2;

}

int main(){

int num;

int areaCode, exchangeCode, lineNumber;

printf("Please entre the phone number.\n");

scanf("%d", &num);

phoneDecompose(num, &areaCode, &exchangeCode, &lineNumber);

printf("The areacode is %d, the exchangecode is %d, the linenumber is %d.\n", areaCode, exchangeCode, lineNumber);

return 0;

}

Task II:

#include <stdio.h>

int Tribonacci(int *n*){ if (*n* <= 0){

printf("Incorrect value.\n");

return -1;

}

 if (*n* == 1){

return 0;

```

    }else if (n == 2 || n == 3){
        return 1;
    }
    return Tribonacci(n - 1) + Tribonacci(n - 2) + Tribonacci(n - 3);
}
int main(){
    int n;
    printf("Please enter the number.\n");
    scanf("%d", &n);
    if (n > 0)
        printf("The value of the term is %d.", Tribonacci(n));
    else
        Tribonacci(n);
    return 0;
}

```

Program output:

Task I:

```

Please entre the phone number.
2125551234
The areacode is 212, the exchangecode is 555, the linenumber is 1234

```

Task II:

```

Please enter the number.
7
The value of the term is 13.

```

```

Please enter the number.
0
Incorrect value.

```

Analysis:

1. Phone Number Decomposition Function

- To modify the value of three variables in the function, we need to use the pointer to set the address of the variables in the main function rather than just rely on the return value.
- Correspondingly, we need to pass the reference about the three variables when calling the function rather than just pass the value to ensure the correct implementation.
- Since we just pass the value of the number, the modification in the function does not affect the value in the main function.

2. Tribonacci Sequence

- We first specify the initial case about the recursion for Tribonacci. There the initial case is when n equals to 1, 2 and 3.
- Then we need to ensure the formula of recursion and make it as the return value to finish the

recursion.

- Note to check the validity of the number n .

Conclusion:

When we need to change the value more than one variable in the function or we want to store the status in the return value and do other incidents in the function body, we need to use the pointers, that is, we need to pass by reference rather than pass by value.

For recursion, we need to specify the initial case. Then we need to find the relationship between the function with itself to ensure the formula. Meanwhile note the recursion does not exceed the limitation of stack.

LAB REPORT

Lab Title	Arrays	Lab No.	06
Student Name	Li Rongyao	Student ID	2024103975
Date	2025/4/7		

Lab description/objectives:

1. Demonstrate understanding of two-dimensional arrays and array manipulation in C programming.
2. Implement the functionality of finding the largest element in each row, calculating the transpose of a square matrix and performing symmetry Judgment as defined in the lab tasks/requirements.
3. Analyze critical aspects.

Source code:

Task I:

```
#include <stdio.h>
```

```
int M;
```

```
void maxRow(int a[M][M],int b[M]){
```

```
    for (int i = 0; i < M; i++){
```

```
        b[i] = a[i][0];
```

```
        for (int j = 1; j < M; j++){
```

```
            if (a[i][j] > b[i]){
```

```
                b[i] = a[i][j];
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

```
int main(){
```

```
    printf("Please enter the size of the square matrix.\n");
```

```
    scanf("%d", &M);
```

```
    int a[M][M], b[M];
```

```
    printf("Please enter the element of the matrix.\n");
```

```
    for (int i = 0; i < M; i++){
```

```
        for (int j = 0; j < M; j++){
```



```
        scanf("%d", &a[i][j]);
    }
}
maxRow(a, b);
for (int i = 0; i < M; i++){
    printf("The largest element in the row %d is %d\n", i, b[i]);
}
return 0;
}
```

Task II:

```
#include <stdio.h>
```

```
int N;
```

```
void transpose(int a[N][N], int b[N][N]){
```

```
    for (int i = 0; i < N; i++){
        for (int j = 0; j < N; j++){
            b[j][i] = a[i][j];
        }
    }
}
```

```
}
```

```
int isSymmetric(int a[N][N]){
```

```
    int ret = 1;
    for (int i = 0; i < N; i++){
        for (int j = i + 1; j < N; j++){
            if (a[i][j] != a[j][i]){
                ret = 0;
                return ret;
            }
        }
    }
    return ret;
}
```

```
}
```

```
int main(){

    printf("Please enter the size of the square matrix.\n");

    scanf("%d", &N);

    int a[N][N], b[N][N];

    printf("Please enter the elements in the matrix.\n");

    for (int i = 0; i < N; i++){

        for (int j = 0; j < N; j++){

            scanf("%d", &a[i][j]);

        }

    }

    transpose(a, b);

    printf("The transpose of the matrix is:\n");

    for (int i = 0; i < N; i++){

        for (int j = 0; j < N; j++){

            printf("%d ", b[i][j]);

        }

        printf("\n");

    }

    if (isSymmetric(a)){

        printf("The matrix is symmetric.\n");

    }else{

        printf("The matrix is not symmetric.\n");

    }

    return 0;

}
```

Program output:

Task I:

```
Please enter the size of the square matrix.
```

```
3
```

```
Please enter the element of the matrix.
```

```
1 3 4
```

```
2 5 5
```

```
8 7 1
```

```
The largest element in the row 0 is 4
```

```
The largest element in the row 1 is 5
```

```
The largest element in the row 2 is 8
```

Task II:

```
Please enter the size of the square matrix.
```

```
3
```

```
Please enter the elements in the matrix.
```

```
1 2 4
```

```
3 4 2
```

```
1 5 6
```

```
The transpose of the matrix is:
```

```
1 3 1
```

```
2 4 5
```

```
4 2 6
```

```
The matrix is not symmetric.
```

```
Please enter the size of the square matrix.
```

```
4
```

```
Please enter the elements in the matrix.
```

```
1 2 4 5
```

```
2 3 2 1
```

```
4 2 5 6
```

```
5 1 6 3
```

```
The transpose of the matrix is:
```

```
1 2 4 5
```

```
2 3 2 1
```

```
4 2 5 6
```

```
5 1 6 3
```

```
The matrix is symmetric.
```

Analysis:

1. Finding the Largest Element in Each Row

- Clarify the column size of the two-dimensional array when passing the reference to the function so that the compiler can ensure the right address.

- Ensure the iterator will not exceed the limit of the size. That is to control the boundary as less than N and begin at 0.

2. Transpose of a Matrix and Symmetry Judge

- Clear the logical sequence when store the transpose of matrix A to B.

- When judging the symmetry, choose the appropriate method to reduce the times of loop. That is just need to check the half of matrix to another half. So we can let the inner loop begin at $i+1$.

Conclusion:

When using the array, we need to ensure the size of it in advance and notice the boundary when iterating the array. When using the high-dimensional array as the parameter of functions, we need to clarify the necessary size to ensure the compiler can find the right continuous address.

LAB REPORT

Lab Title	Sorting Algorithms	Lab No.	07
Student Name	Li Rongyao	Student ID	2024103975
Date	2025/4/14		
Lab description/objectives: 1. Demonstrate understanding of sorting methods in C programming. 2. Implement sorting functions with customizable as defined in the lab tasks/requirements. 3. Analyze critical aspects.			
Source code: Bubble Sort: <pre>#include <stdio.h> #define Asize 10 typedef double Elementype; void printA(Elementype *a, int size) { for (int i = 0; i < size; i++) { printf("%.1f ", a[i]); } printf("\n"); } void reset(Elementype *a, int size){ Elementype b[Asize] = {4.4, 3.3, 2.2, 5.5, 1.1, 6.6, 7.7, 10.0, 9.9, 8.8}; for (int i = 0; i < size; i++){ a[i] = b[i]; } } void swap(Elementype *a, Elementype *b) { Elementype tmp = *a; *a = *b; *b = tmp; } void BubbleSort(Elementype *a, int size, char type) { int end = size - 1; while (end) {</pre>			

```

        for (int i = 0; i < end; i++)
        {
            if (a[i] > a[i + 1])
                swap(&a[i], &a[i + 1]);
        }
        end--;
    }
    if (type == 'd')
    {
        for (int i = 0; i < size / 2; i++)
        {
            swap(&a[i], &a[size - i - 1]);
        }
    }
}

int main()
{
    Elementype a[Asize] = {4.4, 3.3, 2.2, 5.5, 1.1, 6.6, 7.7, 10.0, 9.9, 8.8};
    printf("The elements in the array before sorting is:\n");
    printA(a, Asize);
    BubbleSort(a, Asize, 'a');
    printf("The elements in the array after sorting in ascending order is:\n");
    printA(a, Asize);
    reset(a, Asize);
    BubbleSort(a, Asize, 'd');
    printf("The elements in the array after sorting in descending order is:\n");
    printA(a, Asize);
    reset(a, Asize);
    return 0;
}

Selection Sort:
#include <stdio.h>
#define Asize 10
typedef double Elementype;
void printA(Elementype* a, int size){
    for (int i = 0; i < size; i++){
        printf("%.1f ", a[i]);
    }
    printf("\n");
}

void swap(Elementype* a, Elementype* b){
    Elementype tmp = *a;
    *a = *b;

```

```

    *b = tmp;
}
void reset(Elementtype *a, int size)
{
    Elementtype b[Asize] = {4.4, 3.3, 2.2, 5.5, 1.1, 6.6, 7.7, 10.0, 9.9, 8.8};
    for (int i = 0; i < size; i++)
    {
        a[i] = b[i];
    }
}
void SelectionSort(Elementtype* a, int size, char type){
    int minidx;
    for (int i = 0; i < (size - 1); i++){
        minidx = i;
        for (int j = i + 1; j < size; j++){
            if (a[i] > a[j]){
                minidx = j;
            }
        }
        swap(&a[i], &a[minidx]);
    }
    if (type == 'd'){
        for (int i = 0; i < size / 2; i++){
            swap(&a[i], &a[size - i - 1]);
        }
    }
}
int main(){
    Elementtype a[Asize] = {4.4, 3.3, 2.2, 5.5, 1.1, 6.6, 7.7, 10.0, 9.9, 8.8};
    printf("The elements in the array before sorting is:\n");
    printA(a, Asize);
    SelectionSort(a, Asize, 'a');
    printf("The elements in the array after sorting in ascending order is:\n");
    printA(a, Asize);
    reset(a, Asize);
    SelectionSort(a, Asize, 'd');
    printf("The elements in the array after sorting in descending order is:\n");
    printA(a, Asize);
    reset(a, Asize);
    return 0;
}
Quick Sort:
#include <stdio.h>

```

```
#define Asize 10
typedef double Elementype;
void printA(Elementype *a, int size)
{
    for (int i = 0; i < size; i++)
    {
        printf("%.1f ", a[i]);
    }
    printf("\n");
}
void swap(Elementype *a, Elementype *b)
{
    Elementype tmp = *a;
    *a = *b;
    *b = tmp;
}
void reset(Elementype *a, int size)
{
    Elementype b[Asize] = {4.4, 3.3, 2.2, 5.5, 1.1, 6.6, 7.7, 10.0, 9.9, 8.8};
    for (int i = 0; i < size; i++)
    {
        a[i] = b[i];
    }
}
int Partition(Elementype* a, int low, int high){
    Elementype pivot = a[low];
    while (low < high){
        while (low < high && a[high] >= pivot){
            high--;
        }
        a[low] = a[high];
        while (low < high && a[low] <= pivot){
            low++;
        }
        a[high] = a[low];
    }
    a[low] = pivot;
    return low;
}
void QuickSortA(Elementype* a, int low, int high){
    if (low < high)
    {
        Elementype pivot = Partition(a, low, high);
```



```

        QuickSortA(a, low, pivot - 1);
        QuickSortA(a, pivot + 1, high);
    }
}

void QuickSort(Elementtype* a, int size, char type){
    QuickSortA(a, 0, size - 1);
    if (type == 'd'){
        for (int i = 0; i < size / 2; i++)
        {
            swap(&a[i], &a[size - i - 1]);
        }
    }
}

int main(){
    Elementtype a[Asize] = {4.4, 3.3, 2.2, 5.5, 1.1, 6.6, 7.7, 10.0, 9.9, 8.8};
    printf("The elements in the array before sorting is:\n");
    printA(a, Asize);
    QuickSort(a, Asize, 'a');
    printf("The elements in the array after sorting in ascending order is:\n");
    printA(a, Asize);
    reset(a, Asize);
    QuickSort(a, Asize, 'd');
    printf("The elements in the array after sorting in descending order is:\n");
    printA(a, Asize);
    reset(a, Asize);
    return 0;
}

```

Program output:

```

The elements in the array before sorting is:
4.4 3.3 2.2 5.5 1.1 6.6 7.7 10.0 9.9 8.8
The elements in the array after sorting in ascending order is:
1.1 2.2 3.3 4.4 5.5 6.6 7.7 8.8 9.9 10.0
The elements in the array after sorting in descending order is:
10.0 9.9 8.8 7.7 6.6 5.5 4.4 3.3 2.2 1.1

```

Analysis:

1. Bubble Sort

- Choose the appropriate loop to ensure the smallest number gradually come to the front(if ascending)
- Use another array to perform the descending order.

2. Selection Sort

- Understand the logic of choose the appropriate size number to its position.

3. Quick Sort

- Clear the logic of use the partition and recursion.

- Use another function to perform the descending order.

Conclusion:

When using the Bubble sort, we need to understand its logic that to continuously compare the neighbor two elements until the smallest element come to the front. When using the Selection sort, we need to understand its logic that set the n th smallest number on the n th position. When using the Quick sort, we need to understand how to choose the pivot, how to partition the array and how to use the recursion.

LAB REPORT

Lab Title	Strings	Lab No.	08
Student Name	Li Rongyao	Student ID	2024103975
Date	2025/4/21		

Lab description/objectives:

1. Demonstrate understanding of strings in C programming.
2. Implement the functionality of counting characters/searching a substring as defined in the lab tasks/requirements.
3. Analyze critical aspects.

Source code:

Task I:

```
#include <stdio.h>
#define size 100
char countingstr(char* str, int* max){
    char tmp = str[0];
    char ret = tmp;
    int i = 1;
    *max = 1;
    int tmpmax = *max;
    while (str[i] != '\0'){
        if (str[i] == tmp){
            tmpmax++;
            if (tmpmax > *max)
            {
                *max = tmpmax;
                ret = tmp;
            }
        }else{
            tmp = str[i];
            tmpmax = 1;
        }
        i++;
    }
    return ret;
}
int main() {
    int max = 1;
    char str[size];
    printf("Please enter a string.\n");
```

```
gets(str);
char maxchar = countingstr(str, &max);
printf("The maximum number is %d, the character is %c", max, maxchar);
return 0;
}
Task II:
#include <stdio.h>
#define size 100
int subsearch(char* str, char* substr){
    int ret = -1;
    int i = 0, j = 0, k = 0;
    for (i = 0; str[i] != '\0'; i++)
    {
        if (str[i]==substr[j]){
            if (j == 0)
                k = i;
            j++;
            if (substr[j] == '\0'){
                ret = k;
                break;
            }
            continue;
        }
        j = 0;
    }
    return ret;
}
int main(){
    char str[size], substr[size];
    printf("Please enter the main string.\n");
    gets(str);
    printf("Please enter the substring you want to find.\n");
    gets(substr);
    int pos = subsearch(str, substr);
    if (pos != -1){
        printf("The substring starts at index %d.\n", pos);
    } else {
        printf("Not found.\n");
    }
    return 0;
}
```

Program output:

Task I:

```
Please enter a string.  
aabbbbcc  
The maximum number is 4, the character is b
```

```
Please enter a string.  
ee rttt t  
The maximum number is 3, the character is t
```

Task II:

```
Please enter the main string.  
aabb123cc  
Please enter the substring you want to find.  
123  
The substring starts at index 4.
```

```
Please enter the main string.  
dd eibdf  
Please enter the substring you want to find.  
ei  
The substring starts at index 2.
```

Analysis:

1. Counting Consecutive Repeating Characters

- When writing the function, we need to not only give the information of the maximum but also give the character. So we need to use pointers to modify the max as the maximum and use return value as the character.

- We need to renew the maximum and corresponding character until ending the string.
- Let the variable *maxchar* receive the return value of the function.
- The input should use *gets()* so that the string can receive the space.

2. Substring Searching

- We need to respectively store the position in both main string and substring. So we choose *i, j* to record their position and use *k* to store the starts of substring in main strings.

- If the next character is not the corresponding element in substring, we need to clear the situation and restart the find again so we need let *j* be zero again.

Conclusion:

For the strings, the input and output should choose the appropriate function, such as *printf()*, *scanf()* and *gets()*, *puts()*. And the terminal element in a string is '\0', which can be used as the terminal condition of loop.

LAB REPORT

Lab Title	Data Files	Lab No.	09
Student Name	Li Rongyao	Student ID	2024103975
Date	2025/4/27		

Lab description/objectives:

1. Demonstrate understanding of the data files in C programming.
2. Implement functionality of File Data Filtering and Average Calculation as defined in the lab tasks/requirements.
3. Analyze critical aspects.

Source code:

main.c:

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    FILE *infile;
    int gsize, tmp;
    infile = fopen("testfile.txt", "r");
    if (infile == NULL)
    {
        printf("Failed.\n");
        return 1;
    }
    while ((fscanf(infile, "%d", &gsize)) != EOF)
    {
        int sum = 0;
        int cnt = 0;
        int ave;
        for (int i = 1; i <= gsize; i++)
        {
            fscanf(infile, "%d", &tmp);
            if (tmp % 3 != 0)
            {
                sum += tmp;
                cnt++;
            }
        }
        if (cnt != 0){
            ave = sum / cnt;
```

```
        printf("The average of this group is %d.\n", ave);
    }else{
        printf("This group has no corresponding number.\n");
    }
}
fclose(infile);
return 0;
}
testfile.txt:
4 10 20 30 40
3 50 60 70
5 80 90 100 110 120
```

Program output:

```
The average of this group is 23.
The average of this group is 60.
The average of this group is 96.
```

Analysis:

1. Key Challenges & Solutions

- Learn about the function *fscanf()* to get the data in another file.
- Know that the pointer of *fscanf()* is sequent so it can be used as the terminal condition of the outer loop.

2. Result Validation

- When reading the file, we need to check whether successfully read. If not we need return error information.
- When calculating the average, we need to check whether the denominator is zero.
- Note that break the connection between the files at last.

Conclusion:

When reading the file, we need to check whether the operation is successful and make a error judgement.

When using the nested loop to input the file, the EOF can be used as the terminal condition of outer loop and the function *fscanf()* can be used to sequentially input the data in another file.

LAB REPORT

Lab Title	Pointer Notations of Arrays	Lab No.	10
Student Name	Li Rongyao	Student ID	2024103975
Date	2025/4/28		

Lab description/objectives:

1. Demonstrate understanding of pointer notations of arrays in C programming.
2. Implement sorting functions in pointer notations as defined in the lab tasks/requirements.
3. Analyze critical aspects.

Source code:

BubbleSort:

```
#include <stdio.h>
#define MSize 10
typedef float Elementype;
void bubbleSort(Elementype *arrPtr, int numel, int (*func)(Elementype, Elementype));
int dec(Elementype a, Elementype b);
int inc(Elementype a, Elementype b);
void swap(Elementype *a, Elementype *b);
int main()
{
    Elementype num[MSize] = {4.4, 3.3, 2.2, 5.5, 1.1, 6.6, 7.7, 10.0, 9.9, 8.8};
    bubbleSort(num, MSize, dec);
    for (int i = 0; i < MSize; i++)
    {
        printf("%.1f ", *(num + i));
    }
    printf("\n");
    bubbleSort(num, MSize, inc);
    for (int i = 0; i < MSize; i++)
    {
        printf("%.1f ", *(num + i));
    }
    printf("\n");
    return 0;
}
int dec(Elementype a, Elementype b)
{
    return a < b;
}
```



```

int inc(Elementtype a, Elementtype b)
{
    return b < a;
}

void bubbleSort(Elementtype *arrPtr, int numel, int (*func)(Elementtype, Elementtype))
{
    int end = numel;
    while (end--)
    {
        for (int i = 0; i < end; i++)
        {
            if (func(*(arrPtr + i), *(arrPtr + i + 1)))
            {
                swap((arrPtr + i), (arrPtr + i + 1));
            }
        }
    }
}

void swap(Elementtype *a, Elementtype *b)
{
    Elementtype temp = *a;
    *a = *b;
    *b = temp;
}

SelectionSort:
#include <stdio.h>
#define MSize 10
typedef float Elementtype;
void selectionSort(Elementtype *arrPtr, int numel, int (*func)(Elementtype, Elementtype));
int dec(Elementtype a, Elementtype b);
int inc(Elementtype a, Elementtype b);
void swap(Elementtype *a, Elementtype *b);
int main()
{
    Elementtype num[MSize] = {4.4, 3.3, 2.2, 5.5, 1.1, 6.6, 7.7, 10.0, 9.9, 8.8};
    selectionSort(num, MSize, dec);
    for (int i = 0; i < MSize; i++)
    {
        printf("%.1f ", *(num + i));
    }
    printf("\n");
    selectionSort(num, MSize, inc);
    for (int i = 0; i < MSize; i++)

```

```
{
    printf("%.1f", *(num + i));
}
printf("\n");
return 0;
}
int dec(Elementype a, Elementype b)
{
    return a < b;
}
int inc(Elementype a, Elementype b)
{
    return b < a;
}
void selectionSort(Elementype *arrPtr, int numel, int (*func)(Elementype, Elementype))
{
    for (int i = 0; i < numel; i++)
    {
        int extidx = i;
        for (int j = i + 1; j < numel; j++)
        {
            if (func(*(arrPtr + extidx), *(arrPtr + j)))
            {
                extidx = j;
            }
        }
        swap(arrPtr + i, arrPtr + extidx);
    }
}
void swap(Elementype *a, Elementype *b)
{
    Elementype temp = *a;
    *a = *b;
    *b = temp;
}
```

Program output:

```
10.0 9.9 8.8 7.7 6.6 5.5 4.4 3.3 2.2 1.1
1.1 2.2 3.3 4.4 5.5 6.6 7.7 8.8 9.9 10.0
```

Analysis:**1. Key Challenges & Solutions**

- Understand the pointers to the function as the parameter of another function to use different function in a function.

- Understand method of the offset of the pointer to represent the elements in the array.
- Understand the logic of decreasing and increasing.

2. Result Validation

- Print the element in the array after sorting to ensure the correctness of the result.
- Note the format of output to make the result more readable.

3. Improvement Suggestions

- Use the *Elementtype* to typedef *float* to make the code more sustainable.
- Separate the *swap()* function to make the code more clear.

Conclusion:

When using the pointer notation to represent array elements, we need to understand the offset and the order of taking value and moving the iteration. When using the pointer to the function, we need to understand the logic and the storage of the function in the memory and its usage in other functions.

LAB REPORT

Lab Title	Structures and Unions for Animal Information	Lab No.	11
Student Name	Li Rongyao	Student ID	2024103975
Date	2025/5/12		

Lab description/objectives:

1. Demonstrate understanding of structures and unions in C programming.
2. Implement functionality of creating a structure array of animals as defined in the lab tasks/requirements.
3. Analyze critical aspects.

Source code:

```
#include <stdio.h>
#define L 4
struct Animal
{
    char name[20];
    int animalID;
    char animalCategory;
    union Attribute
    {
        int lifespan;
        float flightSpeed;
    } attribute;
} animal[L];
int inputanimal(int i, int* mnum, int* bnum);
void outputanimal(int i);
int main()
{
    int n;
    int tmp;
    int mnum = 0, bnum = 0;
    printf("Please enter the number of animals.\n");
    if ((tmp = scanf("%d", &n)) < 1 || n > L)
    {
        printf("Error input.\n");
        return 1;
    }
    for (int i = 0; i < n; )
    {
```

```

        if (inputanimal(i, &mnum, &bnum)){
            i++;
        };
    }
    if ((n >= 2) && (mnum == 0 || bnum == 0)){
        printf("Please enter at least one mammal and one bird.\n");
        return 1;
    }
    for (int i = 0; i < n; i++){
        outputanimal(i);
    }
    return 0;
}

int inputanimal(int i, int* mnum, int* bnum){
    printf("Please enter the name of animal %d.\n", i);
    scanf("%19s", animal[i].name);
    printf("Please enter the ID of animal %d.\n", i);
    scanf("%d", &animal[i].animalID);
    printf("Please enter the category of animal %d.\n", i);
    scanf(" %c", &animal[i].animalCategory);
    if (animal[i].animalCategory != 'M' && animal[i].animalCategory != 'B'){
        printf("Error input.\n");
        return 0;
    }
    if (animal[i].animalCategory == 'M')
    {
        printf("Please enter the lifespan of animal %d.\n", i);
        scanf("%d", &animal[i].attribute.lifespan);
        (*mnum)++;
        return 1;
    }
    else if (animal[i].animalCategory == 'B')
    {
        printf("Please enter the flight speed of animal %d.\n", i);
        scanf("%f", &animal[i].attribute.flightSpeed);
        (*bnum)++;
        return 1;
    }
}

void outputanimal(int i){
    printf("The name of animal %d is %s.\n", i, animal[i].name);
    printf("The ID of animal %d is %d.\n", i, animal[i].animalID);
    printf("The category of animal %d is %c.\n", i, animal[i].animalCategory);
}

```

```
if (animal[i].animalCategory == 'M'){  
    printf("The lifespan of animal %d is %d.\n", i, animal[i].attribute.lifespan);  
}else{  
    printf("The flight speed of animal %d is %.2f km/h.\n", i, animal[i].attribute.flightSpeed);  
}  
}
```

Program output:

```
Please enter the number of animals.  
2  
Please enter the name of animal 0.  
bird  
Please enter the ID of animal 0.  
23  
Please enter the category of animal 0.  
B  
Please enter the flight speed of animal 0.  
24  
Please enter the name of animal 1.  
mammal  
Please enter the ID of animal 1.  
25  
Please enter the category of animal 1.  
M  
Please enter the lifespan of animal 1.  
24  
The name of animal 0 is bird.  
The ID of animal 0 is 23.  
The category of animal 0 is B.  
The flight speed of animal 0 is 24.00 km/h.  
The name of animal 1 is mammal.  
The ID of animal 1 is 25.  
The category of animal 1 is M.  
The lifespan of animal 1 is 24.
```

```
Please enter the number of animals.
2
Please enter the name of animal 0.
mammal
Please enter the ID of animal 0.
23
Please enter the category of animal 0.
M
Please enter the lifespan of animal 0.
24
Please enter the name of animal 1.
Mammal
Please enter the ID of animal 1.
25
Please enter the category of animal 1.
M
Please enter the lifespan of animal 1.
23
Please enter at least one mammal and one bird.
```

Analysis:

1. Key Challenges & Solutions

- Separate the procedures of input and output into functions.
- Understand the relationship among variables, unions and structures.
- Initialize a structure variable when creating a structure.

2. Result Validation

- Confirm the category are right.
- Ensure it has at least one mammal and one bird when more than two animals.

3. Improvement Suggestions

- Can confirm the more validation of the name, ID, lifespan and category.
- The format of the output can be optimized.

Conclusion:

The structure can combine different individual variables together and the memory is the sum of them while the union is similar to the structure but they use the same memory. And we need to know how to use the members in structures or unions including using pointers.

LAB REPORT

Lab Title	Dynamic Data Structures	Lab No.	12
Student Name	Li Rongyao	Student ID	2024103975
Date	2025/5/19		

Lab description/objectives:

1. Demonstrate understanding of the usage of dynamic data structures especially the link-list-based stack in C programming.
2. Implement the push, pop, sum functions as defined in the lab tasks/requirements.
3. Analyze the performance of the stack and each function.

Source code:

```
#include <stdio.h>
struct menu
{
    int num;
    struct menu *priorAddr;
};
struct menu *tosp = NULL;
void pushstk();
void poponestk();
void sumstk();
void popstk();
int main()
{
    int opt;
    printf("Option 1: Push Integer to Stack.\n");
    printf("Option 2: Pop Integer from Stack.\n");
    printf("Option 3: Calculate the sum of Stack Integers.\n");
    printf("Option 4: Quit.\n");
    do {
        printf("Enter the number to do corresponding task.\n");
        scanf("%d", &opt);
        switch (opt)
        {
            case 1:
                pushstk();
                break;
            case 2:
                poponestk();
                break;
```



```
        case 3:
            sumstk();
            break;
        default:
            popstk();
            break;
    }
} while (opt != 4);
return 0;
}

void pushstk()
{
    int tmp;
    printf("Please enter the integer you want to push.\n");
    scanf("%d", &tmp);
    struct menu* node = (struct menu *)malloc(sizeof(*tosp));
    if (node == NULL){
        printf("Failed to get the memory.\n");
        return;
    }
    node->num = tmp;
    node->priorAddr = tosp;
    tosp = node;
}

void poponestk()
{
    if (tosp != NULL)
    {
        int tmp = tosp->num;
        struct menu *tmpaddr = tosp->priorAddr;
        free(tosp);
        tosp = tmpaddr;
        printf("The integer %d on the top has been popped.\n", tmp);
    } else {
        printf("Error. The stack is empty.\n");
    }
}

void sumstk(){
    if (tosp == NULL){
        printf("Error. The stack is empty.\n");
        return;
    }
    int sum = 0;
```

```
struct menu *tmpptr = tosp;
while (tmpptr != NULL){
    sum += tmpptr->num;
    tmpptr = tmpptr->priorAddr;
}
printf("The sum of all integers on the stack is %d.\n", sum);
}
void popstk(){
    while (tosp != NULL){
        struct menu *tmpptr = tosp->priorAddr;
        free(tosp);
        tosp = tmpptr;
    }
    printf("The stack has been popped.\n");
}
```

Program output:

```
Option 1: Push Integer to Stack.
Option 2: Pop Integer from Stack.
Option 3: Calculate the sum of Stack Integers.
Option 4: Quit.
Enter the number to do corresponding task.
1
Please enter the integer you want to push.
2
Enter the number to do corresponding task.
1
Please enter the integer you want to push.
3
Enter the number to do corresponding task.
1
Please enter the integer you want to push.
1
Enter the number to do corresponding task.
2
The integer 1 on the top has been popped.
Enter the number to do corresponding task.
3
The sum of all integers on the stack is 5.
Enter the number to do corresponding task.
4
The stack has been popped.
```

Analysis:

1. Key Challenges & Solutions

- Understand the performance of the link-list-based stack and its principle of LIFO.
- Code the push, pop and sum functions based on the theory of stack.

2. Result Validation

- When implementing the pop one integer and sum function, check whether the stack is empty and

show the error message.

- Appropriate method to free all the memory of the whole stack when implementing option 4.

3. Improvement Suggestions

- Can use a method to cope with the situation that the memory has not been allocated rightly after using the *malloc()* function.
- Can optimize the name of the variables.

Conclusion:

To this lab, we need to know the theory of the link-list-based stack and its dynamic data structures.

Further, we need to know the specific performance of its push, pop and other related functions.

Otherwise, we need to learn about the right way to allocate the memory and free it.

LAB REPORT

Lab Title	Bitwise Operation with Command-Line Argument	Lab No.	13
Student Name	Li Rongyao	Student ID	2024103975
Date	2025/5/26		
Lab description/objectives: 1. Demonstrate understanding of the bitwise operation and the usage of command line. 2. Implement the bitwise operation with command-line argument as defined in the lab tasks/requirements. 3. Analyze the error handling and the validity of argument.			
Source code: <pre>#include <stdio.h> #include <string.h> void And(int num1, int num2); void Or(int num1, int num2); void Xor(int num1, int num2); int main(int argc, char* argv[]){ if (argc != 4){ printf("Invalid argument count.\n"); return 1; } int num1, num2; if (!sscanf(argv[1], "%d", &num1) !sscanf(argv[3], "%d", &num2)){ printf("Invalid integer input. Please provide valid integers.\n"); return 1; } if (strcmp(argv[2], "and") && strcmp(argv[2], "or") && strcmp(argv[2], "xor")){ printf("Invalid bitwise operation. Valid operations are 'and', 'or', 'xor'.\n"); return 1; } if (!strcmp(argv[2], "and")) { And(num1, num2); } else if (!strcmp(argv[2], "or")) { Or(num1, num2); } else if (!strcmp(argv[2], "xor")) {</pre>			

```

        Xor(num1, num2);
    }
    return 0;
}

void And(int num1, int num2){
    printf("%d ANDed with %d is %d.\n", num1, num2, num1 & num2);
}

void Or(int num1, int num2){
    printf("%d ORed with %d is %d.\n", num1, num2, num1 | num2);
}

void Xor(int num1, int num2){
    printf("%d XORed with %d is %d.\n", num1, num2, num1 ^ num2);
}

```

Program output:

```

PS D:\code\codeInC> gcc -o D:\code\codeInC\test_25_05_26\bitwise D:\code\codeInC\test_25_05_26\bitwise.c
PS D:\code\codeInC> D:\code\codeInC\test_25_05_26\bitwise 12 and 10
12 ANDed with 10 is 8.
PS D:\code\codeInC> D:\code\codeInC\test_25_05_26\bitwise 12 or 10
12 ORed with 10 is 14.
PS D:\code\codeInC> D:\code\codeInC\test_25_05_26\bitwise 12 xor 10
12 XORed with 10 is 6.
PS D:\code\codeInC> D:\code\codeInC\test_25_05_26\bitwise 12 add 10
Invalid bitwise operation. Valid operations are 'and', 'or', 'xor'.
PS D:\code\codeInC> D:\code\codeInC\test_25_05_26\bitwise a or 5
Invalid integer input. Please provide valid integers

```

Analysis:

1. Key Challenges & Solutions

- Understand the usage of command line and the bitwise operator.
- Use the function `sscanf()` to perform the transformation from the type `char` to `int` in the buffer.

2. Result Validation

- Check whether the count of arguments input by command line is valid by the variable `argc`.
- Use the function `sscanf()` and `strcmp()` to respectively check the validity of the integers and the operation.

3. Improvement Suggestions

- May create a function to replace the function `sscanf()` since I forgot it at the beginning but it is not necessary.
- Optimize the input of command line to repeat the input if error.

Conclusion:

In this experiment, we need to learn about the operations of bitwise and their corresponding operators. And we also need to master the usage of command line to input and corresponding format of the `main()` function.

LAB REPORT

Lab Title	Inheritance and Polymorphism	Lab No.	14
Student Name	Li Rongyao	Student ID	2024103975
Date	2025/6/9		

Lab description/objectives:

1. Demonstrate understanding of Inheritance and Polymorphism in C++ programming.
2. Implement a practical class hierarchy as defined in the lab tasks/requirements.
3. Analyze the usage of inheritance and polymorphism and how it works.

Source code:**Inheritance:**

```
#include <iostream>
using namespace std;
class Person{
public:
    string name;
    int age;
    void introduce(){
        cout << "My name is " << name << "and I'm " << age << " years old." << endl;
    }
};
class Student : public Person{
public:
    int studentID;
    void study()
    {
        cout << "I'm studying hard." << endl;
    }
};
class Teacher : public Person{
public:
    string subject;
    void teach()
    {
        cout << "I'm teaching " << subject << "." << endl;
    }
};
int main(){
    Student st1;
```

```
st1.age = 18, st1.name = "Li", st1.studentID = 2024103975;
Teacher tch1;
tch1.age = 40, tch1.name = "Zhao", tch1.subject = "Maths";
st1.introduce(), st1.study();
tch1.introduce(), tch1.teach();
return 0;
}

Polymorphism:
#include <iostream>
using namespace std;
class Person{
public:
    string name;
    int age;
    void introduce()
    {
        cout << "My name is " << name << "and I'm " << age << " years old." << endl;
    }
    virtual void work() = 0;
};
class Student : public Person{
public:
    int studentID;
    void study()
    {
        cout << "I'm studying hard." << endl;
    }
    virtual void work() override{
        cout << "I'm a student and I'm working on my studies." << endl;
    }
};
class Teacher : public Person{
public:
    string subject;
    void teach()
    {
        cout << "I'm teaching " << subject << "." << endl;
    }
    virtual void work() override{
        cout << "I'm a teacher and I'm working on teaching." << endl;
    }
};
int main(){
```

```
Person *p1 = new Student();
Person *p2 = new Teacher();
p1->work(), p2->work();
delete p1, delete p2;
return 0;
}
```

Program output:

Inheritance:

```
PS D:\code\codeInCpp> D:\code\codeInCpp\test_25_06_09\Inheritance.e
My name is Liand I'm 18 years old.
I'm studying hard.
My name is Zhaoand I'm 40 years old.
I'm teaching Maths.
```

Polymorphism:

```
PS D:\code\codeInCpp> D:\code\codeInCpp\test_25_06_09\polymorphism.exe
I'm a student and I'm working on my studies.
I'm a teacher and I'm working on teaching.
```

Analysis:

1. Key Challenges & Solutions

- Understand the syntax of inheritance and polymorphism and their corresponding practices.
- Understand the base class and subclass in inheritance and virtual function and dynamic binding in polymorphism.

2. Result Validation

- Note the access in the different class.
- Use the base class pointer to point to the object of subclass and then invoke the function in the subclass.
- Note free the corresponding memory of the pointer.

3. Improvement Suggestions

- The access in the class can be modified to ensure the security.

Conclusion:

For inheritance, we need to understand that it aims at improving the percentage of reuse and constructing the relationship among classes. For polymorphism, we need to understand that it aims at unifying the connector to the subclasses in the base class and making the invocation more valid.

LAB REPORT

Lab Title	STL map container	Lab No.	15
Student Name	Li Rongyao	Student ID	2024103975
Date	2025/6/16		

Lab description/objectives:

1. Demonstrate understanding of the STL map container and its function in cpp.
2. Implement the steps of fruit price management as defined in the lab tasks/requirements.
3. Analyze critical functions and performance about map container.

Source code:

```
#include <iostream>
#include <map>
using namespace std;
void Insertingfruit(map<string, double>& fruitPrices){
    string tmpname;
    double tmpprices;
    cout << "Enter fruit name: ";
    cin >> tmpname;
    cout << "Enter price: ";
    cin >> tmpprices;
    fruitPrices.insert({tmpname, tmpprices});
}
void Printingfruit(map<string, double>& fruitPrices){
    for (auto i = fruitPrices.begin(); i != fruitPrices.end(); i++){
        cout << i->first
            << " : "
            << i->second
            << endl;
    }
}
void Searchingfruit(map<string, double>& fruitPrices){
    string name;
    cout << "Please enter the name of the fruit you want to search."
        << endl;
    cin >> name;
    auto tmp = fruitPrices.find(name);
    if (tmp != fruitPrices.end()){
        cout << "The price of "
            << name
```

```
<< " is "
<< tmp->second
<< "."
<< endl;
} else {
    cout << "The fruit is not in the list."
    << endl;
}
}
void ModifyingPrice(map<string, double>& fruitPrices){
    string name;
    double newprice;
    cout << "Please enter the fruit you want to modify the price and its new price."
        << endl;
    cin >> name >> newprice;
    auto tmp = fruitPrices.find(name);
    if (tmp != fruitPrices.end()){
        tmp->second = newprice;
        cout << "The price has been successfully modified."
            << endl;
    } else {
        cout << "Error. The fruit cannot be found for price modification."
            << endl;
    }
}
void Deletingfruit(map<string, double>& fruitPrices){
    string name;
    cout << "Please enter the fruit you want to delete."
        << endl;
    cin >> name;
    auto tmp = fruitPrices.find(name);
    if (tmp != fruitPrices.end()){
        fruitPrices.erase(tmp);
        cout << "The fruit has been successfully deleted."
            << endl;
    } else {
        cout << "Error. The fruit cannot be found for deletion."
            << endl;
    }
}
int main(){
    map<string, double> fruitPrices;
    int num;
```

```
cout << "Please enter the number of initial elements in the fruit list."
    << endl;
cin >> num;
for (int i = 0; i < num; i++){
    Insertingfruit(fruitPrices);
}
cout << "The fruit list is:"
    << endl;
Printingfruit(fruitPrices);
Searchingfruit(fruitPrices);
ModifyingPrice(fruitPrices);
Deletingfruit(fruitPrices);
cout << "The modified fruit list is:"
    << endl;
Printingfruit(fruitPrices);
return 0;
}
```

Program output:

```
Please enter the number of initial elements in the fruit list.
3
Enter fruit name: apple
Enter price: 8
Enter fruit name: banana
Enter price: 3
Enter fruit name: orange
Enter price: 4
The fruit list is:
apple : 8
banana : 3
orange : 4
Please enter the name of the fruit you want to search.
banana
The price of banana is 3.
Please enter the fruit you want to modify the price and its new price.
apple 5
The price has been successfully modified.
Please enter the fruit you want to delete.
orange
The fruit has been successfully deleted.
The modified fruit list is:
apple : 5
banana : 3
```

```
Please enter the number of initial elements in the fruit list.
3
Enter fruit name: apple
Enter price: 5
Enter fruit name: banana
Enter price: 3
Enter fruit name: orange
Enter price: 4
The fruit list is:
apple : 5
banana : 3
orange : 4
Please enter the name of the fruit you want to search.
watermelon
The fruit is not in the list.
Please enter the fruit you want to modify the price and its new price.
bear
4
Error. The fruit cannot be found for price modification.
Please enter the fruit you want to delete.
bear
Error. The fruit cannot be found for deletion.
The modified fruit list is:
apple : 5
banana : 3
orange : 4
```

Analysis:

1. Key Challenges & Solutions

- Understand the various functions in the map container and their corresponding usage.
- Use appropriate iterator to iterate through the container and use the type *auto* to perform.

2. Result Validation

- Confirm passing reference to the function so that the map in the main function can be modified correctly.
- Print the error message when the fruit is not in the list.

3. Improvement Suggestions

- Can check the data type when entering the number of initial elements.

Conclusion:

In cpp, the map container can be used to correspond key to the value and provides various function to insert, delete, modify, search in the container.